

KUBERNETES

Kubernetes is an open-source container orchestration platform used to automate the deployment, scaling, and management of containerized applications.

- It helps manage containers across multiple servers efficiently.
- Kubernetes provides high availability, load balancing, automatic scaling, and self-healing capabilities.

Kubernetes Cluster

- A Kubernetes Cluster is a set of nodes that run containerized applications.
- It consists of a Control Plane (Master Node) and one or more Worker Nodes.
- The Control Plane manages the cluster.
- Worker Nodes run the application workloads inside Pods.

Pod Definition

- A Pod is the smallest unit in Kubernetes.
- It contains one or more containers.
- Pods run applications inside a Kubernetes cluster.

Example:

A Pod can contain an Nginx container that serves web pages.

Service Types in Kubernetes

1. ClusterIP

- Default service type.
- Exposes the application only within the cluster.

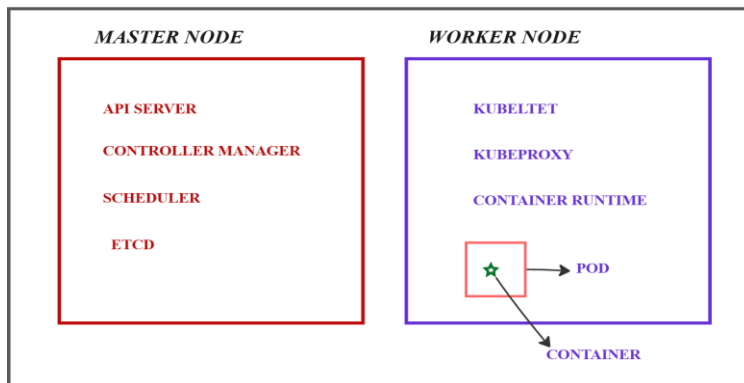
2. NodePort

- Exposes the application on a specific port of each worker node.
- Accessible using NodeIP:NodePort.

3. LoadBalancer

- Exposes the application through a cloud provider load balancer.
- Provides an external IP or DNS name.

KUBERNETES ARCHITECTURFE

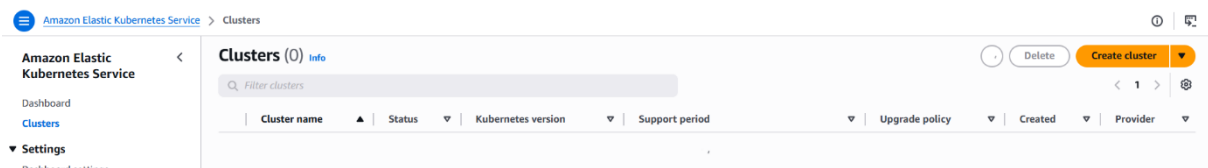


miro

HANDS-ON FOR KUBERNETES

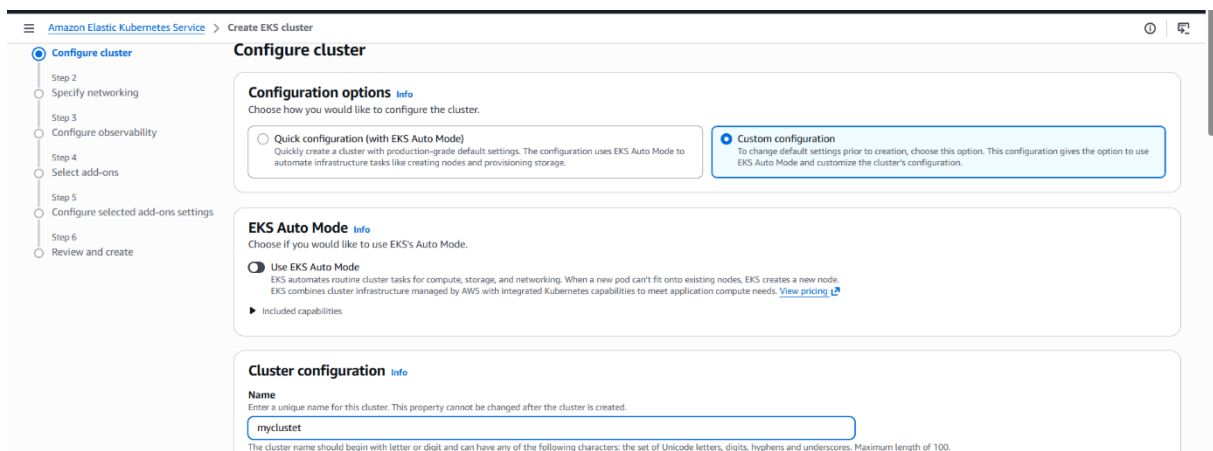
Screenshot: Open Amazon EKS

- Open the **AWS Management Console**.
- Search for **EKS**.
- Select **Elastic Kubernetes Service (EKS)**.
- Click **Create Cluster**.



Screenshot: Configure Cluster Name

- Select **Custom Configuration**.
- Enter the **Cluster Name**.



Screenshot: Select Cluster IAM Role

- Click **Create New Role**.
- Create a new IAM role.
- Select the **created IAM role** from the **Cluster IAM Role** dropdown.
- Click **Next** to continue.

Cluster IAM role

To access other AWS services and resources your Kubernetes control plane needs an IAM role. Choose a role with the right permissions for your Kubernetes control plane.

AmazonEKSClusterRole

Create new role

Search

EKS Kubernetes control plane service-assumable roles

AmazonEKSClusterRole
arn:aws:iam::301804611986:role/AmazonEKSClusterRole

AWSServiceRoleForAmazonEKS
arn:aws:iam::301804611986:role/aws-service-role/eks.amazonaws.com/AWSServiceRoleForAmazonEKS

Screenshot: Configure Cluster Access

- Select **EKS API and ConfigMap**.
- Click **Next**.

Cluster authentication mode [Info](#)

Configure which source the cluster will use for authenticated IAM principals.

- ☐ EKS API
The cluster will source authenticated IAM principals only from EKS access entry APIs.
- ☒ EKS API and ConfigMap
The cluster will source authenticated IAM principals from both EKS access entry APIs and the aws-auth ConfigMap.

Screenshot: Configure Networking

- Select the **Security Group**.
- Select **Public** access.
- Click **Create**.
- EKS cluster creation started successfully.

Additional security groups [Info](#)

EKS automatically creates a cluster security group on cluster creation to facilitate communication between worker nodes and control plane. Optionally, choose additional security groups to apply to the EKS-managed Elastic Network Interfaces that are created in your control plane subnets. To create a new security group, go to the corresponding page in the [VPC console](#).

Select security groups

Clear selected security groups

Filter security groups

☒ sg-01fd0fe3f335a0753 | jai

☐ sg-0230b4c40073e70b6 | launch-wizard-20
launch-wizard-20 created 2026-05-25T10:50:38.906Z

Cluster endpoint access [Info](#)

Configure access to the Kubernetes API server endpoint.

- ☒ Public
The cluster endpoint is accessible from outside of your VPC. Worker node traffic will leave your VPC to connect to the endpoint.
- ☐ Public and private
The cluster endpoint is accessible from outside of your VPC. Worker node traffic to the endpoint will stay within your VPC.
- ☐ Private
The cluster endpoint is only accessible through your VPC. Worker node traffic to the endpoint will stay within your VPC.

EKS Pod Identity (2)

< 1 >

Add-on name	▲ IAM role ↗	▼ Service account
external-dns	Not set	external-dns
vpc-cni	Not set	aws-node

Cancel

Previous

Create

- Click **Create**.
- The EKS cluster enters the **Creating** state.

Screenshot: Launch EC2 Instance

Instances (1/1) Info

Last updated less than a minute ago

Connect

Instance state

Actions

Launch instances

Saved filter sets

Choose filter set

Find Instance by attribute or tag (case-sensitive)

Instance state = running

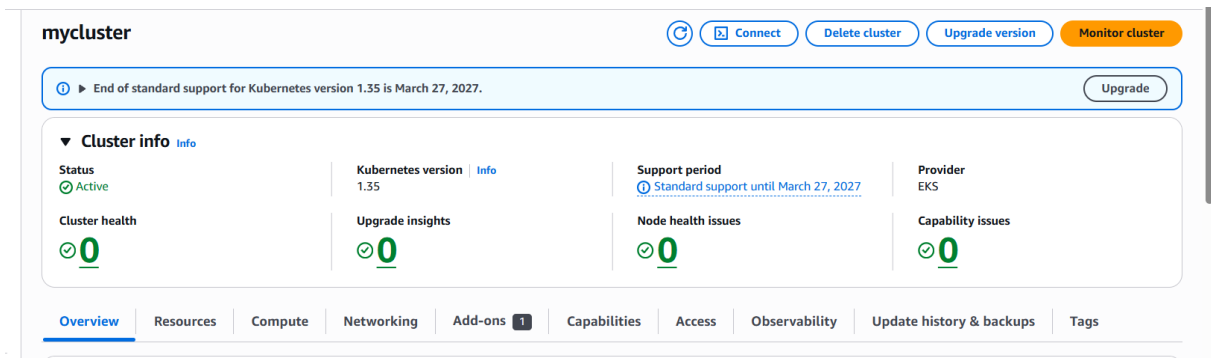
Clear filters

☒	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elas
☒	eks-access	i-0be8034c67b94fa2e	Running	m7i-flex.large	Initializing	ap-south-1b	ec2-52-66-178-112.ap-...	52.66.178.112	-

- Wait for the cluster creation to complete.
- Verify that the cluster status changes from **Creating** to **Active**.

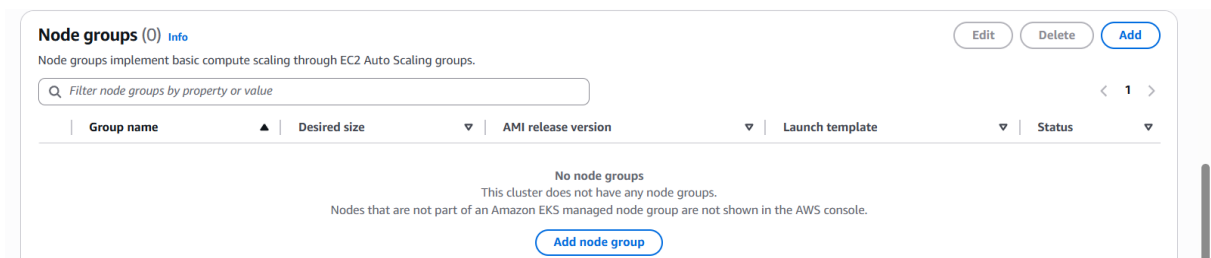
Screenshot: Open Compute Tab

- Click on the **EKS Cluster**.
- Select the **Compute** tab.



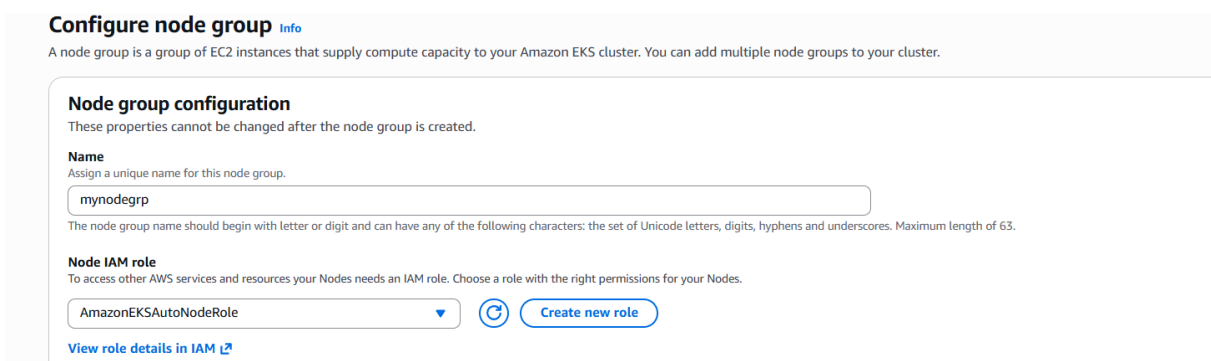
Screenshot: Add Node Group

- Click **Add Node Group**.



Screenshot: Configure Node Group

- Enter the **Node Group Name**.
- Select the **Node IAM Role**.
- Click **Next** to continue.



Screenshot: Configure Node Group Instance

- Select the **AMI Type**.
- Select **On-Demand** capacity.

- Select the **Available Instance Type**.
- Click **Next** to continue.

Set compute and scaling configuration

Node group compute configuration

These properties cannot be changed after the node group is created.

AMI type [Info](#)
Select the EKS-optimized Amazon Machine Image for nodes.

Amazon Linux 2023 (x86_64) Standard (AL2023_x86_64_STANDARD)

Capacity type
Select the capacity purchase option for this node group.

On-Demand

Instance types [Info](#)
Select instance types you prefer for this node group.

Q Enter an instance type

m7i-flex.large
vCPU: 2 vCPUs Memory: 8 GiB Network: Up to 12.5 Gigabit Max ENI: 3 Max IPs: 30

Screenshot: Node Group Active

- Wait for the node group creation to complete.
- Verify that the **Node Group Status** changes to **Active**.

Cluster info

- Status: Active
- Kubernetes version: 1.35
- Support period: Standard support until March 27, 2027
- Provider: EKS
- Cluster health: 0
- Upgrade insights: 0
- Node health issues: 0
- Capability issues: 0

Nodes (2)

Node name	Instance type	Compute	Managed by	Created	Status	CPU usage	Memory usage	Ephemeral storage usage
ip-172-31-31-244.ap-south-1.compute.internal	m7i-flex.large	Node group	mynodegrp	a minute ago	Ready	32%	17%	15%
ip-172-31-44-210.ap-south-1.compute.internal	m7i-flex.large	Node group	mynodegrp	a minute ago	Ready	12%	9%	15%

Node groups (1)

Group name	Desired size	AMI release version	Launch template	Status
mynodegrp	2	1.35.5-20260529	-	Active

Screenshot: Verify Worker Nodes in EC2

- Go to the **EC2 Dashboard**.
- Click **Instances**.
- Verify that **2 Worker Node instances** are available.

Instances (2/3)

Instance state: running

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP	IPv6 IPs
i-02cbdc3c4953b209d	Running	m7i-flex.large	Initializing	ap-south-1c	ec2-13-200-138-220.ap...	13.200.138.220	-	-	
i-044853c0af74153dd	Running	m7i-flex.large	Initializing	ap-south-1a	ec2-43-205-140-30.ap...	43.205.140.30	-	-	
eks-access	i-0be8034c67b94fa2e	Running	m7i-flex.large	3/3 checks passed	ap-south-1b	ec2-52-66-178-112.ap...	52.66.178.112	-	-

Screenshot: Configure AWS CLI

- Connect to the **eks-access** EC2 instance.
- Run the command: `aws configure`
- Enter the **AWS Access Key ID**.
- Enter the **AWS Secret Access Key**.
- Enter the **Default Region**.
- Press **Enter** for the output format or set it if required.
- AWS CLI configured successfully.

Instances (1/3) Info

Connect Instance state Actions Launch instances

Instance state: running

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP	IPv6 IPs
	i-02cbdc3c4955b209d	Running	m7i-flex.large	Initializing	ap-south-1c	ec2-13-200-138-220.ap...	13.200.138.220	-	-
	i-044853c0af74153dd	Running	m7i-flex.large	Initializing	ap-south-1a	ec2-43-205-140-30.ap...	43.205.140.30	-	-
eks-access	i-0be8034c67b94fa2e	Running	m7i-flex.large	3/3 checks passed	ap-south-1b	ec2-52-66-178-112.ap...	52.66.178.112	-	-

```
[ec2-user@ip-172-31-8-99 ~]$ aws configure
AWS Access Key ID [None]: AKIAUMRHU5GJGEMIPVMK
AWS Secret Access Key [None]: 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
Default region name [None]: Default output format [None]: [ec2-user@ip-172-31-8-99 ~]$
[ec2-user@ip-172-31-8-99 ~]$
```

Screenshot: Update kubeconfig

- Connect to the **eks-access** EC2 instance.
- Run the following command:

```
aws eks --region ap-south-1 update-kubeconfig --name mycluster
```

- The kubeconfig file is updated successfully.
- The EC2 instance is now configured to communicate with the EKS cluster.

```
[ec2-user@ip-172-31-8-99 ~]$ aws eks --region ap-south-1 update-kubeconfig --name mycluster
Added new context arn:aws:eks:ap-south-1:301804611986:cluster/mycluster to /home/ec2-user/.kube/config
[ec2-user@ip-172-31-8-99 ~]$
```

Screenshot: Verify Worker Nodes

- Run `kubectl get nodes`.
- Worker nodes are displayed successfully.

```
[ec2-user@ip-172-31-8-99 ~]$ kubectl get nodes
NAME                                STATUS    ROLES    AGE    VERSION
ip-172-31-31-244.ap-south-1.compute.internal Ready    <none>   9m1s   v1.35.5-eks-3385e9b
ip-172-31-44-210.ap-south-1.compute.internal Ready    <none>   9m2s   v1.35.5-eks-3385e9b
[ec2-user@ip-172-31-8-99 ~]$
```

i-0be8034c67b94fa2e (eks-access)

PublicIPs: 52.66.178.112 PrivateIPs: 172.31.8.99

Screenshot: Create Pod YAML File

- Run `vi pod.yaml`.
- Paste the Pod configuration.
- Press **Esc**.
- Type `:wq!`.
- Press **Enter** to save and exit the file.

```
[ec2-user@ip-172-31-8-99 ~]$ vi pod.yaml
[ec2-user@ip-172-31-8-99 ~]$
```

```
kind: Pod
metadata:
  name: my-app-pod
  labels:
    app: my-app
spec:
  containers: # ✅ must be 'containers'
  - name: my-container
    image: httpd:latest
    ports:
    - containerPort: 80
~
```

Screenshot: Create Pod

- Run `kubectl apply -f pod.yaml`.
- Pod created successfully.

```
[ec2-user@ip-172-31-8-99 ~]$ kubectl apply -f pod.yaml
pod/my-app-pod created
```

Screenshot: Verify Pod

- Run `kubectl get pods`.
- Pod information displayed successfully.

```
[ec2-user@ip-172-31-8-99 ~]$ kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
my-app-pod    1/1     Running   0           24s
[ec2-user@ip-172-31-8-99 ~]$
```

Screenshot: Create Service YAML File

- Run `vi service.yaml`.
- Paste the Service configuration.
- Press **Esc**.
- Type `:wq!`.
- Press **Enter** to save and exit the file.

```
pod.yaml
[ec2-user@ip-172-31-8-99 ~]$ vi service.yaml
[ec2-user@ip-172-31-8-99 ~]$
```

i-0be8034c67b94fa2e (eks-access)

```
apiVersion: v1
kind: Service
metadata:
  name: my-app-service
spec:
  selector:
    app: my-app
  type: NodePort
  ports:
    - protocol: TCP
      port: 80      # Service port
      targetPort: 80 # Pod container port
      nodePort: 30007 # NodePort [must be between 30000-32767]
```

Screenshot: Create Service

- Run `kubectl apply -f service.yaml`.
- Service created successfully.

```
[ec2-user@ip-172-31-8-99 ~]$ kubectl apply -f service.yaml
service/my-app-service created
[ec2-user@ip-172-31-8-99 ~]$
```

Screenshot: Verify Service

- Run `kubectl get svc`.
- Service information displayed successfully.

```
[ec2-user@ip-172-31-8-99 ~]$ kubectl get services
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
kubernetes          ClusterIP   10.100.0.1     <none>         443/TCP          50m
my-app-service      NodePort    10.100.80.236  <none>         80:30007/TCP     96s
[ec2-user@ip-172-31-8-99 ~]$
```

Screenshot: Copy Worker Node Public IP

- Go to the **EC2 Dashboard**.
- Select the **Worker Node** instance.
- Copy the **Public IPv4 Address**.

Instance summary for i-02cbdc3c4953b209d [Info](#)

Updated less than a minute ago

Instance ID

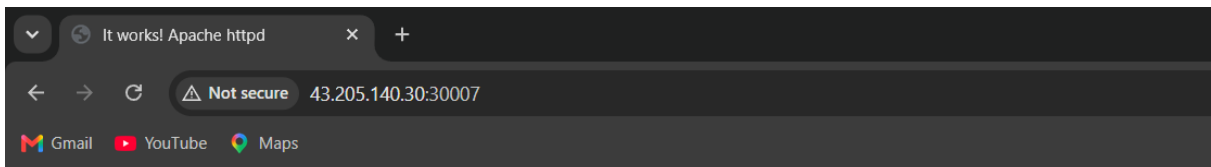
 i-02cbdc3c4953b209d

Public IPv4 address

 [13.200.138.220](#) | [open address](#) 

Screenshot: Access Application

- Open a **new browser tab**.
- Paste the **Worker Node Public IP** followed by **:30007**.
- Press **Enter**.
- The **HTTPD welcome page** is displayed successfully.



It works!



It works!

Screenshot: Verify Pod Details

- Run `kubectl get pods -o wide`.
- Pod details are displayed successfully.

```
[ec2-user@ip-172-31-8-99 ~]$ kubectl get pod -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE                                           NOMINATED NODE   READINESS GATES
my-app-pod    1/1     Running   0           27m   172.31.39.250   ip-172-31-44-210.ap-south-1.compute.internal   <none>            <none>
```

Screenshot: Delete Pod

- Run `kubectl delete pod my-app-pod`.
- Pod deleted successfully.
- Run `kubectl delete service my-app-service`.
- Service deleted successfully.

```
[ec2-user@ip-172-31-8-99 ~]$ kubectl delete pod my-app-pod
pod "my-app-pod" deleted from default namespace
[ec2-user@ip-172-31-8-99 ~]$ kubectl delete svc my-app-service
service "my-app-service" deleted from default namespace
[ec2-user@ip-172-31-8-99 ~]$
```

Screenshot: Create Deployment YAML File

- Run `vi deployment.yaml`.
- Paste the **Deployment configuration** with **Replicas** and **Nginx image**.
- Press **Esc**.

- Type `:wq!`.
- Press **Enter** to save and exit the file.

```
[ec2-user@ip-172-31-8-99 ~]$
[ec2-user@ip-172-31-8-99 ~]$ vi deployment.yaml
[ec2-user@ip-172-31-8-99 ~]$
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app-deployment
spec:
  replicas: 5
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: my-container
          image: nginx:latest
          ports:
            - containerPort: 80
~
```

Screenshot: Create Deployment

- Run `kubectl apply -f deployment.yaml`.
- Deployment created successfully.

```
[ec2-user@ip-172-31-8-99 ~]$ kubectl apply -f deployment.yaml
deployment.apps/my-app-deployment created
[ec2-user@ip-172-31-8-99 ~]$
```

Screenshot: Verify Pod Details

- Run `kubectl get pods -o wide`.
- Pod details are displayed successfully.

```
[ec2-user@ip-172-31-8-99 ~]$ kubectl get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
my-app-deployment-58f59c99c-p82p7	1/1	Running	0	36s	172.31.43.245	ip-172-31-44-210.ap-south-1.compute.internal	<none>	<none>
my-app-deployment-58f59c99c-qftrj	1/1	Running	0	36s	172.31.29.80	ip-172-31-31-244.ap-south-1.compute.internal	<none>	<none>
my-app-deployment-58f59c99c-swbd1	1/1	Running	0	36s	172.31.32.30	ip-172-31-44-210.ap-south-1.compute.internal	<none>	<none>
my-app-deployment-58f59c99c-vxdtv	1/1	Running	0	36s	172.31.43.106	ip-172-31-44-210.ap-south-1.compute.internal	<none>	<none>
my-app-deployment-58f59c99c-xl99z	1/1	Running	0	36s	172.31.17.75	ip-172-31-31-244.ap-south-1.compute.internal	<none>	<none>

```
[ec2-user@ip-172-31-8-99 ~]$
```

i-0be8034c67b94fa2e (eks-access)

Screenshot: Delete Pods

- Run `kubectl delete pod <pod-id1> <pod-id2>`.
- Pods deleted successfully.

```
[ec2-user@ip-172-31-8-99 ~]$ kubectl delete pod my-app-deployment-58f59c99c-p82p7 my-app-deployment-58f59c99c-swbd1
pod "my-app-deployment-58f59c99c-p82p7" deleted from default namespace
pod "my-app-deployment-58f59c99c-swbd1" deleted from default namespace
[ec2-user@ip-172-31-8-99 ~]$
```

Screenshot: Verify Pods After Deletion

- Run `kubectl get pods -o wide`.
- Press Enter.
- Two new Pods are created automatically by the ReplicaSet.
- Pod details are displayed successfully.

```
[ec2-user@ip-172-31-8-99 ~]$ kubectl get pod -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE                                NOMINATED NODE   READINESS GATES
my-app-deployment-58f59c99c-f2fpw   1/1     Running   0           13s   172.31.39.125   ip-172-31-44-210.ap-south-1.compute.internal   <none>           <none>
my-app-deployment-58f59c99c-qfrtj   1/1     Running   0           119s  172.31.29.80    ip-172-31-31-244.ap-south-1.compute.internal   <none>           <none>
my-app-deployment-58f59c99c-vxdcv   1/1     Running   0           119s  172.31.43.106   ip-172-31-44-210.ap-south-1.compute.internal   <none>           <none>
my-app-deployment-58f59c99c-xl99z   1/1     Running   0           119s  172.31.17.75    ip-172-31-31-244.ap-south-1.compute.internal   <none>           <none>
my-app-deployment-58f59c99c-zprv7   1/1     Running   0           13s   172.31.37.57    ip-172-31-44-210.ap-south-1.compute.internal   <none>           <none>
[ec2-user@ip-172-31-8-99 ~]$
```

Screenshot: Create Service YAML File

- Run `vi service.yaml`.
- Paste the Service configuration with type LoadBalancer.
- Press Esc.
- Type `:wq!`.
- Press Enter to save and exit the file.
- Service YAML file created successfully.

```
[ec2-user@ip-172-31-8-99 ~]$ vi service.yaml
[ec2-user@ip-172-31-8-99 ~]$
```

```
apiVersion: v1
kind: Service
metadata:
  name: my-app-service
spec:
  selector:
    app: my-app
  type: LoadBalancer
  ports:
    - protocol: TCP
      port: 80      # Service port
      targetPort: 80 # Pod container port
```

Screenshot: Create Service

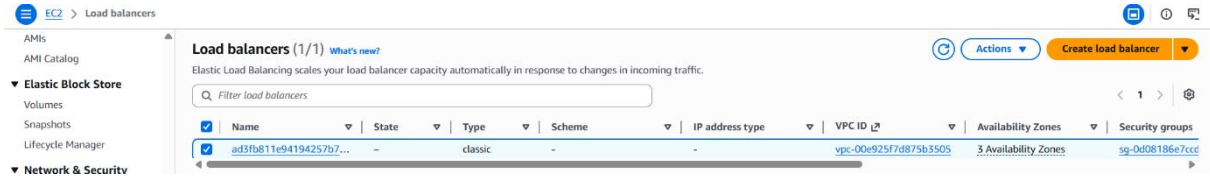
- Run `kubectl apply -f service.yaml`.
- Service created successfully.

```
[ec2-user@ip-172-31-8-99 ~]$ kubectl apply -f service.yaml
service/my-app-service created
[ec2-user@ip-172-31-8-99 ~]$
```

Screenshot: Verify Load Balancer

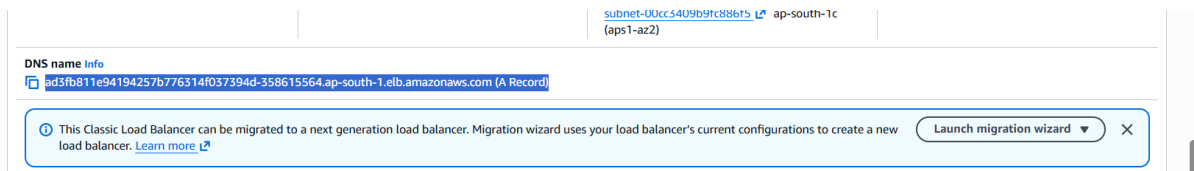
- Open the AWS Management Console.

- Navigate to **EC2**.
- Click **Load Balancers** under **Load Balancing**.
- Verify that the Load Balancer is created successfully.
- Load Balancer details are displayed successfully.



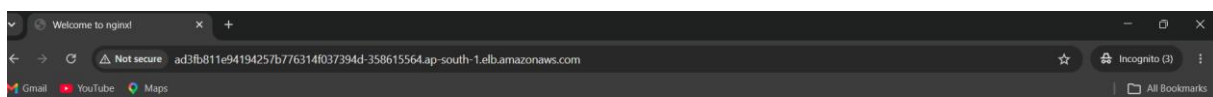
Screenshot: Copy Load Balancer DNS Name

- Select the created Load Balancer.
- Copy the **DNS name** from the Details section.
- DNS name copied successfully.



Screenshot: Access Nginx Application

- Open a new browser tab.
- Paste the Load Balancer DNS name in the address bar.
- Press Enter.
- The Nginx welcome page is displayed successfully.



Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to f5.com/nginx.

Thank you for using nginx.